UNIVERSIDAD POLITECNICA DE ATLAUTLA

Carrera: Ing. en Tecnologías de la Información e Innovación Digital

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Ing. Javier García Lara

Alumna: Veronica Juliette Lopez Perez

Grupo: 171

Tema: 2.4 Algoritmos de búsqueda ciega (BFS /DFS) Bohm, cap. 3

Código + Comparativa. Implementar BFS y DFS en Python para resolver un laberinto o puzle Documentar. representación del grafo, complejidad temporal/espacial, camino solución, comparación (tiempo, nodos expandidos) y conclusión sobre idoneidad según problema.

Problema: Encontrar un camino desde un nodo de inicio hasta un nodo meta en un entorno de cuadrícula (laberinto) utilizando estrategias de búsqueda ciega (no informada).

Representación del Grafo: La cuadrícula se modela como un grafo no dirigido donde cada celda o intersección libre es un nodo V y los pasillos transitables entre celdas vecinas (arriba, abajo, izquierda, derecha) son las aristas E con costo uniforme.

Análisis de Complejidad Temporal y Espacial

BFS (Breadth-First Search - Búsqueda en Anchura):

Complejidad Temporal: O (V + E) (donde V es el número de vértices y E el de aristas), ya que recorre todos los nodos y aristas adyacentes en el peor de los casos.

Complejidad Espacial: O(V), debido a que almacena todos los nodos del nivel actual en la memoria (la cola).

DFS (Depth-First Search - Búsqueda en Profundidad):

Complejidad Temporal: O (V + E), recorre los nodos hasta agotar las ramas.

Complejidad Espacial: O(V) en el peor de los casos, dependiendo de la profundidad máxima del árbol o grafo almacenado en la pila.

TABLA COMPARATIVA

MÉTRICA / CARACTERÍSTICA	BFS (ANCHURA)	DFS (PROFUNDIDAD)
Camino Solución Encontrado	Corto y óptimo (en aristas sin peso)	Profundo (no garantiza el más corto)
Nodos Expandidos	Menor o igual (explora por capas)	Variable (puede explorar caminos largos antes)
Uso de Memoria	Mayor (almacena niveles completos)	Menor (almacena solo la rama actual)
Completitud	Sí (encuentra solución si existe)	No necesariamente en grafos infinitos o con ciclos

CONCLUSION

BFS es el algoritmo ideal para resolver laberintos o problemas de navegación donde se requiere encontrar el camino mínimo o más corto en grafos no ponderados, a pesar de consumir mayor cantidad de memoria.

DFS es preferible cuando la memoria disponible es muy limitada y el objetivo principal es simplemente encontrar una salida o solución viable sin importar si es la más corta u óptima.

BIBLIOGRAFIA

- Russell, S., & Norvig, P. (2020). Inteligencia Artificial: Un enfoque moderno (4ª ed.). Pearson.
- García Serrano, A. (2016). Inteligencia Artificial: Fundamentos, práctica y aplicaciones (2ª ed.). RC Libros.
- Bohm, R. A. (2021). The Fundamentals of Search Algorithms. Nova Science.
- Wirsansky, E. (2020). Hands-On Genetic Algorithms with Python. Packt.
- Kristensen, T. S. (2021). Artificial Intelligence: Models, Algorithms and Applications. Bentham.
- Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann.
- Gemini Team, Google. (2023). Gemini: A family of highly capable multimodal models. arXiv preprint arXiv:2312.11805. https://doi.org/10.48550/arXiv.2312.11805
- Anthropic. (2024). The Claude 3 model family: Opus, Sonnet, Haiku. Anthropic Research. https://www.anthropic.com/news/claude-3-family